OFFLINE AI DOCUMENT SUMMARIZER – MOBILE APP

ICT 410: Supervised Project

Sudarshan Prasad ID: 20220162

1. System Requirements Analysis

1.1 Problem Definition

Sumz helps to summarize documents. The user selects a file, asks for a summary, and receives an answer from an AI model running on the phone.

The problem

- Long documents take time to read.
- Users may only need the main points or a simple explanation.
- Sending private documents to online AI tools may be unsuitable.
- An internet connection may not always be available.

The proposed solution

The app reads PDF, DOCX, and TXT files. It uses a local model to summarize the text and answer questions. It saves chat messages on the device so users can read them later.

Project limits

Document processing is local, but the current login uses Google, Firebase and SupaBase. A fully offline launch is not yet established. Scanned PDFs need OCR, which the app does not provide. Long documents are shortened before they reach the model.

1.2 Stakeholder Identification

Stakeholder	Main need
Document reader	Choose a file and understand it quickly.
Student or researcher	Find key points and ask questions about a document.
Developer	Maintain the screens, model, file readers, and database.

Supporting services

The phone supplies file access and local storage. Google, Firebase and Supabase support sign-in and profile storage.

1.3 Functional Requirements

Functional requirements describe what the app should do. The table separates working features from features that still need work.

ID	Requirement
FR01	Sign in with Google.
FR02	Choose PDF, DOCX, or TXT files.
FR03	Read text from a selected file.
FR04	Summarize, list key points, or explain text.
FR05	Answer questions about the loaded file.
FR06	Save and reopen chat history.
FR07	Start a new chat and delete old chats.
FR08	Copy an answer and sign out.
FR09	Create and recall memories automatically.

1.4 Non-Functional Requirements

Area	Requirement
Privacy	Process document text on the device.
Usability	Use clear labels and simple actions.
Reliability	Keep saved chats after restart.
Performance	Show loading and answer progress.
Offline use	Run extraction and inference locally.
Maintainability	Keep major services separate.

Practical limits

Response speed depends on the Device CPU, RAM and the amount of text.

The model uses CPU processing. The document context is limited to about 28,000 characters, and each answer is limited to 1,024 generated tokens.

1.5 User Requirements

User need	Expected experience
Choose a document	See a file picker for PDF, DOCX, and TXT.
Know what is loaded	See the selected filename in Chat.
Get a quick answer	Use Summarize, Key points, or Explain.
Ask a follow-up	Type a question and see the reply appear.
Read past work	Open history and select a saved chat.
Handle mistakes	See a clear error and retry where possible.
Control saved content	Delete a chat or clear stored memories.

Information users need

AI model can generate wrong information, user to cross check with valid information. Users should know when a document is shortened, when the model is still loading, and when an answer cannot be completed. The current truncation event is logged but is not clearly shown as a user warning.

Reopened chats contain saved messages, but not the original document text. Signing out also leaves local history on the device. These limits should be explained clearly.

Example task

Open a short PDF, choose Summarize, ask for the key points, then reopen the chat from history.

2. Phase 2: System Design and Prototyping

2.1 Login and Home

Link: FigmaWireFrame

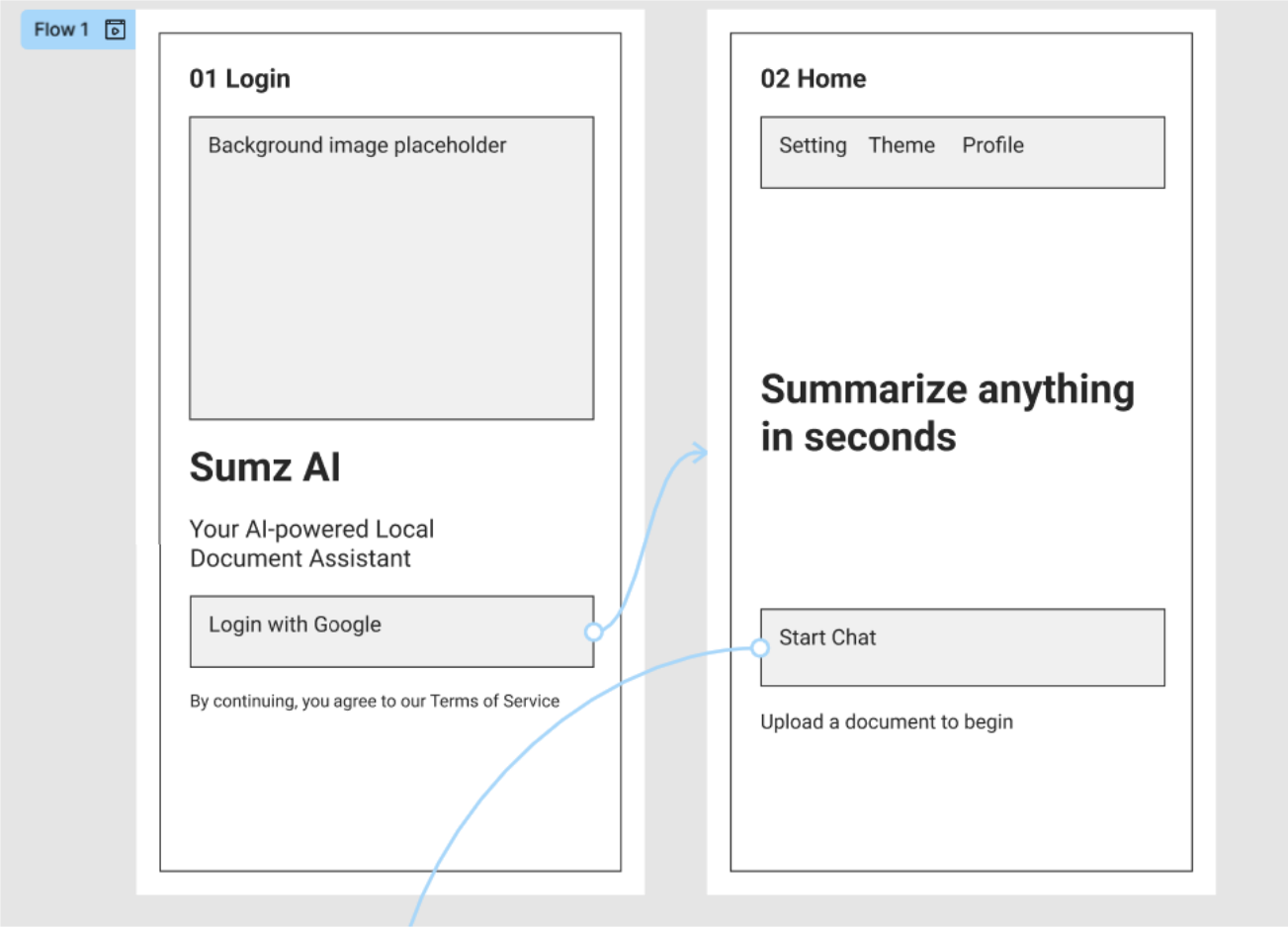


Figure 2.1 Login and Home screen skeletons.

Login shows the app name and Google sign-in. Home shows the theme and profile controls, a short message, and Start Chat.

2.2 Chat and History

Chat has different views for choosing a file, asking questions, and opening saved history. These are states of one screen, not three separate routes.

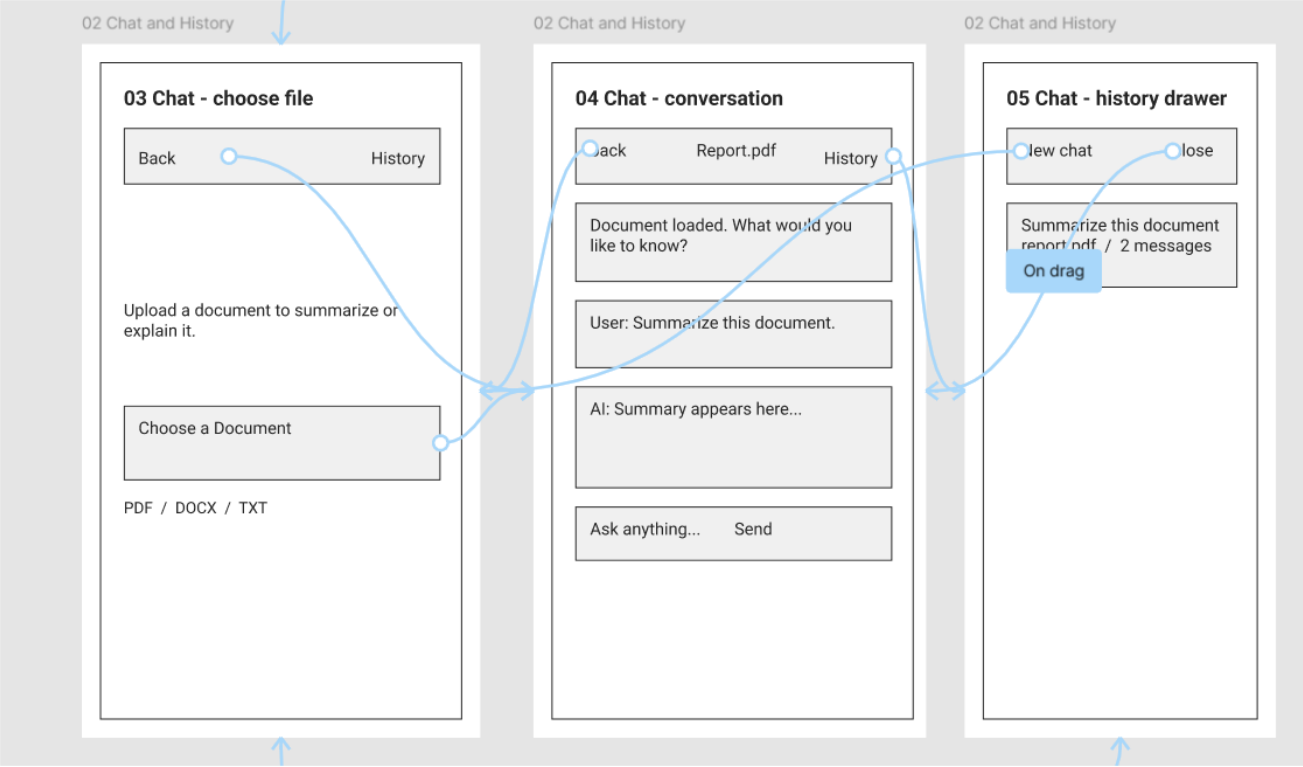


Figure 2.2 File selection, conversation, and history drawer.

View	Main action
Choose file	Select a PDF, DOCX, or TXT document.
Conversation	Use a quick action or type a question.
History drawer	Open an old chat, start a new one, or delete a chat.

2.3 Memory and Settings

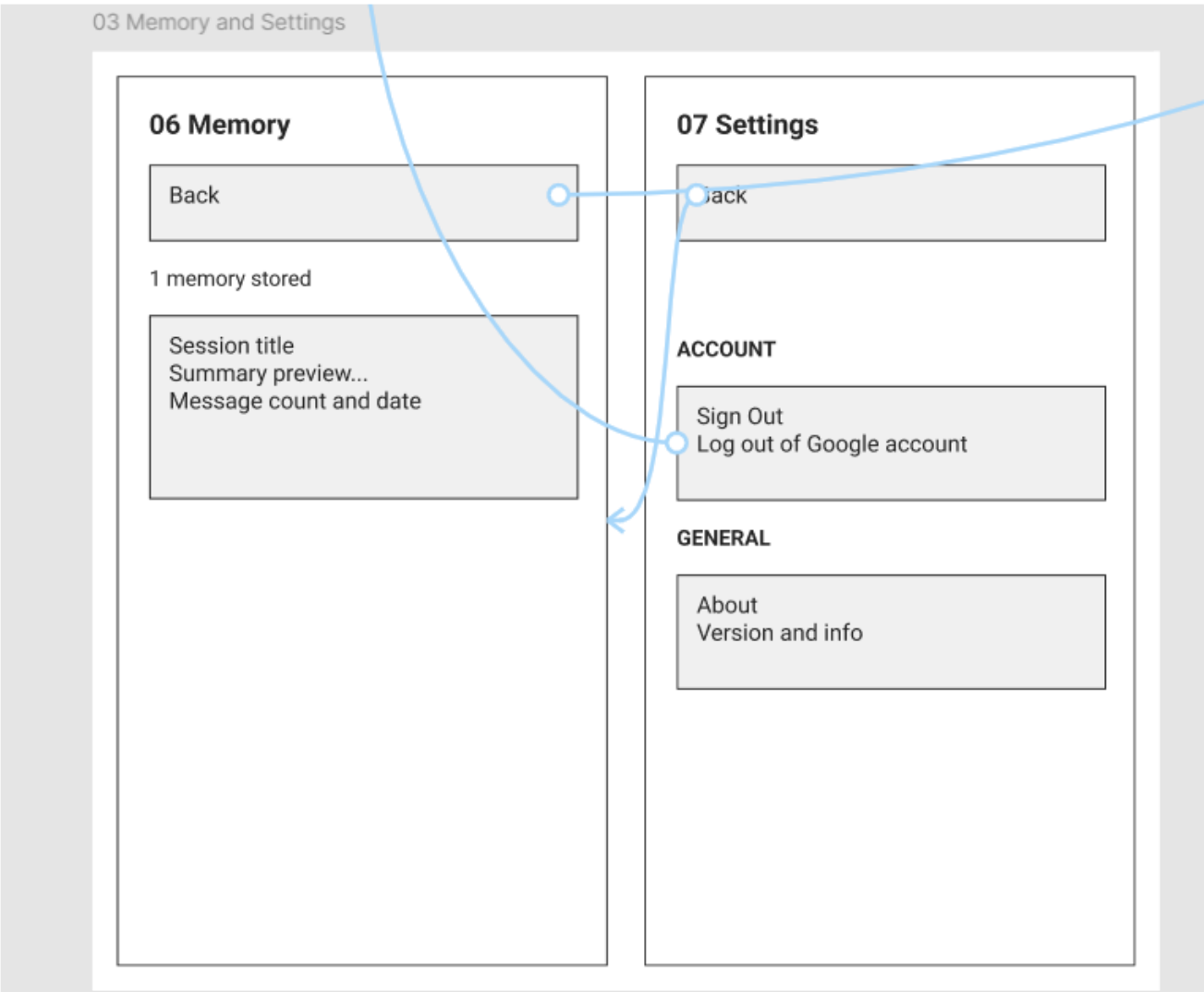


Figure 2.3 Memory and Settings screen skeletons.

Memory shows a saved summary, its title, message count, and date. It also supports an empty state and Delete all.

Settings contains Sign Out and About. About is a placeholder. Both screens are registered in navigation.

3. System Design

3.1 System Architecture

The app is built with React Native and Kotlin - Android first, IOS Later.

Part	What it does
Screens	Collect user actions and show results.
Document service	Select a file and read its text.
Local AI service	Load the model and generate an answer.
SQLite database	Save sessions, messages, and optional memories.
Firebase account service	Handle identity and remote profile details.

Main path

User → selected file → text reader → local model → answer → saved chat.

Offline boundary

The document reader and AI model run locally.

Platform support

Android has the custom PDF, DOCX, and TXT readers.

3.2 System Components and Modules

Module	Main job
DocumentService	Open the picker, detect file type, and shorten long text.
Native file readers	Extract text from PDF, DOCX, and TXT files.
LlamaService	Load model.gguf and stream generated answers.
DatabaseService	Create, read, update, and delete local records.
FirebaseService	Save or read the online user profile.
ModelTemplate	Tell the model to focus on the document.
Parameter	Set response length and generation options.

Screen support

ThemeSwitcher controls light and dark colors. UploadingModal shows file-reading progress. ToastNotification shows short feedback messages. Message options allow an answer to be copied.

Important file-reader limits

PDF reading does not include OCR. DOCX reading covers the main document body, and table text needs correction. TXT reading tries UTF-8 and then ISO-8859-1.

3.3 System Workflows

Step	What happens
1	The user signs in and opens Chat from Home.
2	The app loads the local model and opens SQLite.
3	The user chooses a supported document.
4	The app extracts text and shortens it if needed.
5	The user asks for a summary or enters a question.
6	The app saves the question and builds the model input.
7	The answer appears as the local model generates it.
8	The completed answer is saved in the chat.

Other actions

New chat clears the current screen state. History restores saved messages. Deleting a chat asks for confirmation.

When something fails

Cancelling the picker returns to the screen. An unreadable file shows an error. Model loading has a retry action. Generation failure adds an error message; the question may already be saved.

Restoring history does not restore the original document text.

3.4 Use Case Design

The main actor is the person using the app. The phone file picker and Google/Firebase services support the flow.

Use case	User action	Result
Select a file	Choose a supported document.	Readable text is loaded.
Get a summary	Choose Summarize.	The model gives a short answer.
Ask a question	Type a document question.	The model replies in Chat.
Read history	Open the drawer and select a chat.	Saved messages appear.
Delete a chat	Long-press a chat and confirm.	The session is removed.
Copy an answer	Open message options and copy.	Text enters the clipboard.
Sign out	Choose Sign Out.	The app returns to Login.

Example use case

A student selects lecture notes, asks for the key points, and reads the answer. They then ask for a simple explanation of one topic. Both messages and completed answers are saved.

Conditions

The model must be loaded, and the file must contain readable text. The app blocks blank questions and another request while an answer is being generated.

3.5 Data Flow Design

The app uses two separate data paths: document processing and account management.

From	To	Data
File picker	Native reader	Selected file URI and bytes.
Native reader	Chat state	Extracted document text.
Chat state	Local model	Document text, chat history, and question.
Local model	Chat screen	Answer tokens and final answer.
Chat screen	SQLite	Session details and messages.
Google/Firebase Auth	Login screen	User identity and token.
Login screen	Firestore	Name, email, photo URL, and timestamps.

What stays in memory

The extracted document text is held in the current Chat screen. It is not saved as a document record in SQLite.

What is saved

SQLite stores the filename with the session, plus user questions and completed answers. Firestore stores account profile data.

3.6 Security and Access-Control Design

The app uses Google and Firebase for sign-in. Its local history is stored on the device.

Area	Current design	Limit
Document privacy	Local extraction and inference.	Login still uses online services.
Local history	Saved in byte.db.	No account owner ID in local tables.
Account cache	User and token in AsyncStorage.	No explicit secure-vault storage is shown.
SQL input	Values use query placeholders.	Other data checks still matter.
Android backup	allowBackup is false.	This is not database encryption.
Remote profile	Stored under users/{uid}.	Firestore rules are not supplied here.

Sign-out behavior

Sign-out clears cached account fields, but it does not delete local chats. Another account on the same installation is not separated by the current database design.

Recommended improvements

- Protect tokens with platform secure storage.
- Define account ownership and local deletion rules.
- Limit file size before extraction.
- Remove sensitive content from production logs.

4. Database Design

4.1 Database Requirements

The database keeps the user’s conversation history on the phone. It is named byte.db and uses SQLite.

Requirement	Reason
Unique session IDs	Keep each chat separate.
Messages linked to sessions	Load the right conversation.
Ordered messages	Show questions and answers in sequence.
Recent-session list	Help users find their latest work.
Deletion	Let users remove unwanted history.
One memory per session	Avoid duplicate current memory summaries.

Current scope

The local database has sessions, messages, and memories. It also attempts to create a full-text search index.

Missing capabilities

There is no saved source-document text, per-account history ownership, history expiry rule, or database size limit.

4.2 Entity Identification

Entity	Meaning	Example
Session	One saved conversation.	A chat about report.pdf.
Message	One user question or AI answer.	Please summarize this report.
Memory	One optional saved summary of a session.	A short recap with a title and count.
User profile	Online account details in Firestore.	UID, name, email, and photo URL.

Document information

A document is not a separate database entity in the current app. The session stores its filename. The extracted text stays in the current screen state.

Summary information

A document summary normally appears as an assistant message. The memories table is a separate feature and is not automatically filled by the current Chat screen.

Search data

messages_fts is a search structure built from message data.

4.3 Entity-Relationship Diagram (ERD)

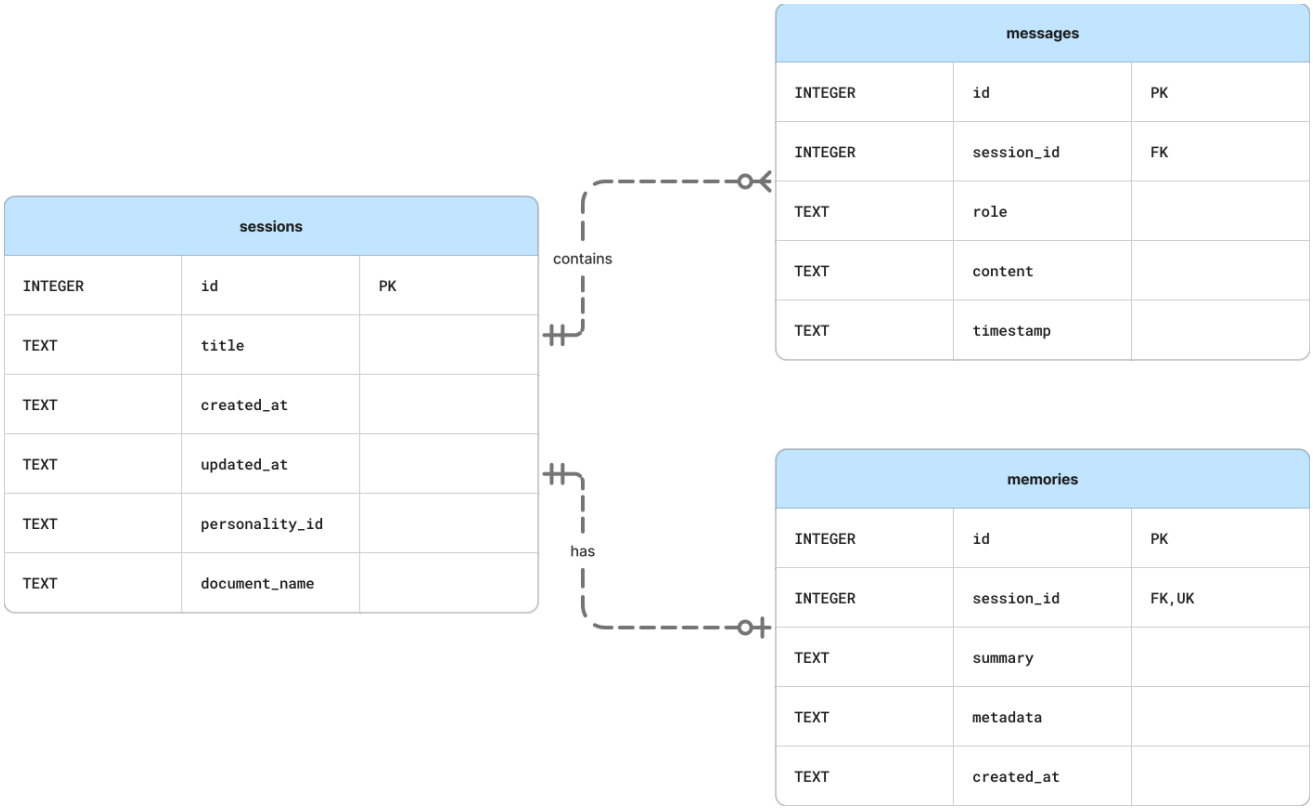


Figure 4.1 Local database relationships.

- One session can have many messages.
- One session can have zero or one memory.
- Each message and memory refers to one session.

4.4 Database Schema

Table	Fields
sessions	id, title, created_at, updated_at, personality_id, document_name
messages	id, session_id, role, content, timestamp
memories	id, session_id, summary, metadata, created_at

Main field rules

Field	Rule
messages.role	Must be user or assistant.
Child session_id	Required link to sessions.id.
memories.session_id	Unique: one memory per session.
personality_id and document_name	Optional text fields.
Dates and times	Stored as ISO text by the service.
memories.metadata	JSON text with session title and message count.

document_name is added during initialization by an ALTER TABLE statement. Indexes help sort recent sessions and memories, and load messages by session and time.

message_count is calculated when sessions are listed.

4.5 Relationships and Cardinality

Cardinality describes how many records can be connected.

Relationship	Count	Meaning
Session → Messages	1 to many	A session may have no messages or several messages.
Session → Memory	1 to zero or one	A session may have one stored memory.
User → Local sessions	Not defined	No UID is stored on the session.
Document → Sessions	Not defined	A filename is stored, not a document ID.

Deletion

Messages and memories declare ON DELETE CASCADE. This is meant to remove child rows when a session is deleted. The service does not explicitly enable SQLite foreign keys.

Memory replacement

Saving a memory again uses INSERT OR REPLACE. It replaces the previous memory for that session instead of keeping several versions. Its row ID and creation time may change.

No many-to-many link

The current local schema has no many-to-many relationship.

4.6 Normalization

Normalization means storing each fact in a suitable place, so the same information does not need to be changed in many rows.

Design choice	Benefit
Keep session details in sessions	The title is not copied into every message.
Keep one turn in each message row	Chats can grow without adding new columns.
Link children by session_id	Records can be joined through a stable ID.
Calculate message_count	The session table does not keep a second count to update.

Normal forms

Sessions and messages use single-column keys and separate their main facts. Under the current rules, they are consistent with third normal form: their non-key fields describe the row identified by the key.

Memory metadata

Memory metadata copies the session title and message count into JSON text. These values can become old if the session changes.

If the screen needs current values, it should read the title and count from the original tables. The full-text search index is a deliberate copy used to support searching.

4.7 Data Integrity and Constraints

Data integrity means keeping records valid and connected correctly.

Rule	Current protection or issue
Every row has an ID	Primary keys identify rows.
Required values exist	NOT NULL is used, but empty text is still allowed.
Roles are valid	A CHECK allows only user or assistant.
One memory per session	A UNIQUE constraint limits duplicates.
Children have a valid parent	Foreign keys are declared; enforcement must be checked.
Writes stay consistent	Message save and session-time update are separate calls.
Metadata is readable	JSON is written by the service.

Recommended checks

- Enable foreign keys and test session deletion.
- Use a transaction for related writes.
- Check database changes with versioned migrations.
- Test invalid roles, missing parents, and broken metadata.

Search limit

FTS5 setup is optional, but searchMessages always uses it.